

11246062 邱偉豪、11246064 張庭語、11246084 鍾百陶、11246085 張家瑜

資管三乙

專題學習進度檢核報告

KERAS大神歸位:深度學習全面進化!用 PYTHON 實作CNN、
RNN、GRU、LSTM、GAN、VAE、TRANSFORMER

指導教授：蒯思齊

2026/02/02

WEEK3

以少量資料集從頭訓練 一個卷積神經網路

實作任務與資料集設定

任務說明：

- 影像分類任務：貓 vs 狗
- 二元分類問題 → 使用 sigmoid + binary crossentropy

資料集概況

- 總共 5,000 張影像
 - 貓：2,500
 - 狗：2,500
- 資料切分方式：
 - 訓練集：2,000
 - 驗證集：1,000
 - 測試集：2,000

將資料集分成3個子資料集，架構如下：

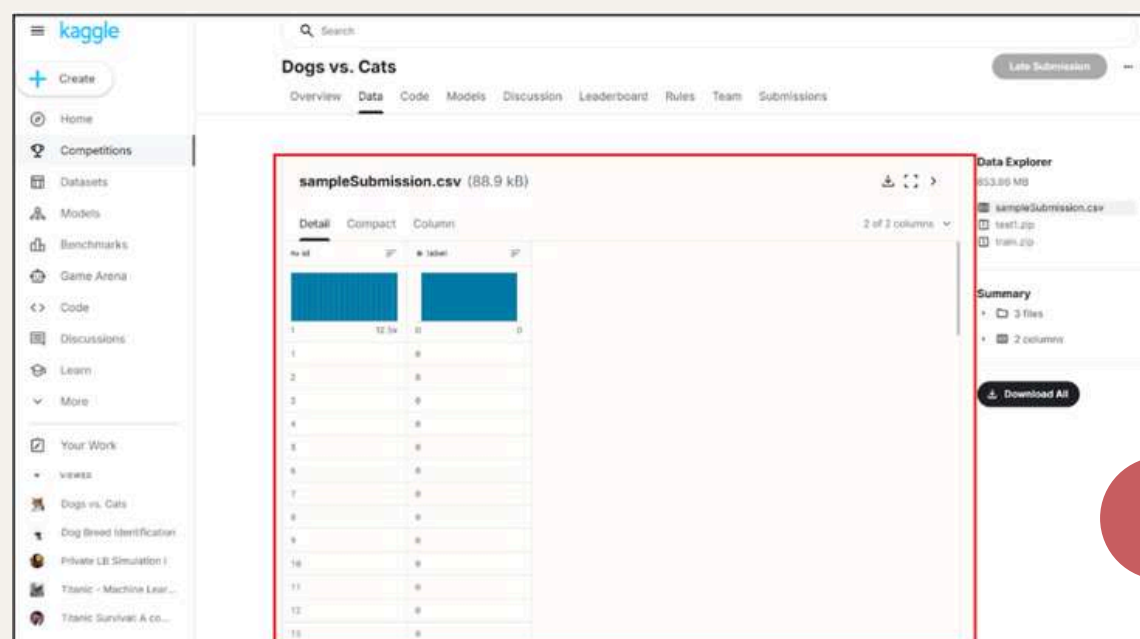
```
cats_vs_dogs_small/  
...train/ ← 訓練集  
.....cat/ ← 包含 1000 張貓的圖片  
.....dog/ ← 包含 1000 張狗的圖片  
...validation/ ← 驗證集  
.....cat/ ← 包含 500 張貓的圖片  
.....dog/ ← 包含 500 張狗的圖片  
...test/ ← 測試集  
.....cat/ ← 包含 1000 張貓的圖片  
.....dog/ ← 包含 1000 張狗的圖片
```

資料量其實不大！！！！

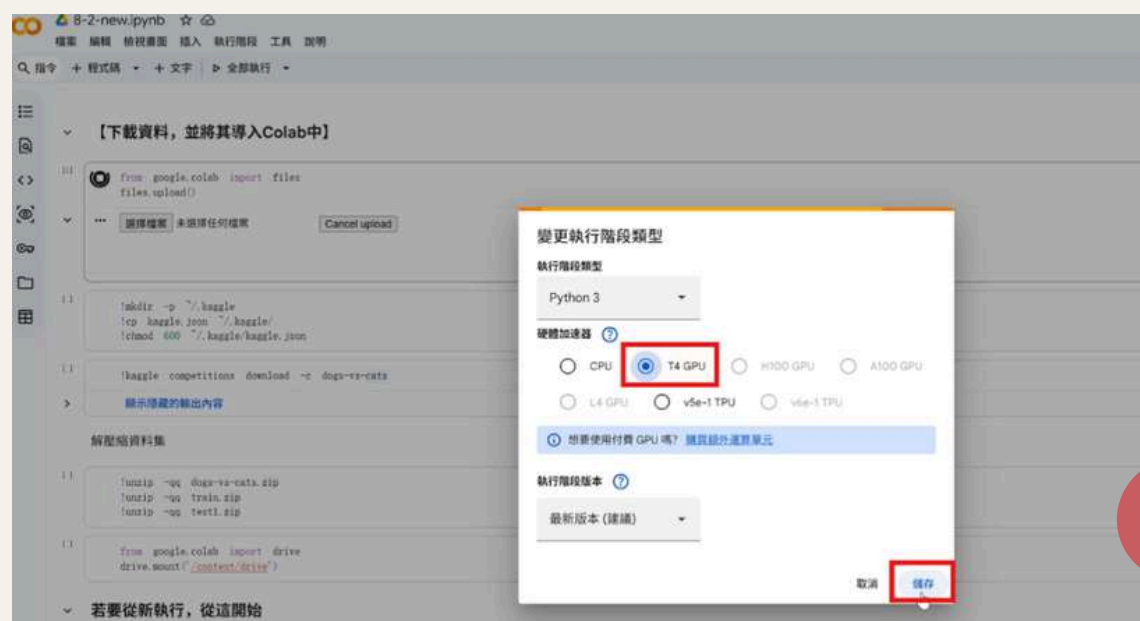
以少量資料集從頭訓練 一個卷積神經網路

資料取得與環境準備

- 資料集來源：Kaggle (Dogs vs Cats)
- 透過 Kaggle API 下載至 Colab
- 使用 **GPU** 作為訓練環境
- 將資料整理成：
 - train / validation / test
 - 並以資料夾結構對應類別



取得Kaggle資料集



使用GPU進行訓練

以少量資料集從頭訓練 一個卷積神經網路

建立神經網路

因為將處理更大的影像、更複雜的問題，因此需要擴大模型的規模，實際做法是組合更多的Conv2D（2D卷積層） + MaxPooling2D（最大池化層）。

```
[7] 10 秒
✓
from tensorflow import keras # 從 TensorFlow 導入 Keras 模組。
from tensorflow.keras import layers # 從 tensorflow.keras 導入 layers 模組，用於構建神經網路層。

inputs = keras.Input(shape=(180,180,3)) # 定義模型的輸入層，指定輸入圖片的形狀為 180x180 像素，3 個顏色通道（RGB）。

x = layers.Rescaling(1./255)(inputs) # 將輸入圖片的像素值從 0-255 縮放到 0-1 範圍，進行數據正規化。
x = layers.Conv2D(filters=32,kernel_size=3,activation='relu')(x) # 第一個卷積層：32 個濾波器，卷積核大小 3x3，使用 ReLU 啟動函數。
x = layers.MaxPooling2D(pool_size=2)(x) # 第一個最大池化層：池化窗口大小 2x2，用於下採樣。
x = layers.Conv2D(filters=64,kernel_size=3,activation='relu')(x) # 第二個卷積層：64 個濾波器，卷積核大小 3x3，使用 ReLU 啟動函數。
x = layers.MaxPooling2D(pool_size=2)(x) # 第二個最大池化層。
x = layers.Conv2D(filters=128,kernel_size=3,activation='relu')(x) # 第三個卷積層：128 個濾波器，卷積核大小 3x3，使用 ReLU 啟動函數。
x = layers.MaxPooling2D(pool_size=2)(x) # 第三個最大池化層。
x = layers.Conv2D(filters=256,kernel_size=3,activation='relu')(x) # 第四個卷積層：256 個濾波器，卷積核大小 3x3，使用 ReLU 啟動函數。
x = layers.MaxPooling2D(pool_size=2)(x) # 第四個最大池化層。
x = layers.Conv2D(filters=256,kernel_size=3,activation='relu')(x) # 第五個卷積層：256 個濾波器，卷積核大小 3x3，使用 ReLU 啟動函數。
x = layers.Flatten()(x) # 將卷積層的輸出展平為一維向量，以便連接到全連接層。
outputs = layers.Dense(1,activation='sigmoid')(x) # 輸出層：一個神經元，使用 Sigmoid 啟動函數，用於二元分類（貓或狗）。
model = keras.Model(inputs=inputs,outputs=outputs) # 創建 Keras 模型，指定輸入和輸出。
```

使用Rescaling層開始，主要是“調整整數輸入像素質的range”，從原始範圍0~255，調整至0~1。

```
[7] 10 秒
✓
from tensorflow import keras # 從 TensorFlow 導入 Keras 模組。
from tensorflow.keras import layers # 從 tensorflow.keras 導入 layers 模組，用於構建神經網路層。

inputs = keras.Input(shape=(180,180,3)) # 定義模型的輸入層，指定輸入圖片的形狀為 180x180 像素，3 個顏色通道（RGB）。

x = layers.Rescaling(1./255)(inputs) # 將輸入圖片的像素值從 0-255 縮放到 0-1 範圍，進行數據正規化。
x = layers.Conv2D(filters=32,kernel_size=3,activation='relu')(x) # 第一個卷積層：32 個濾波器，卷積核大小 3x3，使用 ReLU 啟動函數。
x = layers.MaxPooling2D(pool_size=2)(x) # 第一個最大池化層：池化窗口大小 2x2，用於下採樣。
x = layers.Conv2D(filters=64,kernel_size=3,activation='relu')(x) # 第二個卷積層：64 個濾波器，卷積核大小 3x3，使用 ReLU 啟動函數。
```


以少量資料集從頭訓練 一個卷積神經網路

資料預處理流程

核心流程：

1. 讀取 JPEG 圖片
2. 解碼成 RGB 像素
3. 轉為浮點數張量
4. `resize` 成 180×180
5. 組成 batch (32 張)

這些不用自己處理！

這些步驟在Keras中，使用`image_dataset_from_directory()`函式可全部完整處理好！

```
[10] from tensorflow.keras.utils import image_dataset_from_directory # 從 tensorflow.keras.utils 導入 image_dataset_from_directory 函數，用於從目錄載入圖片資料集。

train_dataset = image_dataset_from_directory( # 載入訓練資料集。
    new_base_dir / 'train', # 指定訓練資料的目錄。
    image_size=(180,180), # 將圖片調整為 180x180 像素。
    batch_size=32 # 設定每個批次的圖片數量為 32。
)

validation_dataset = image_dataset_from_directory( # 載入驗證資料集。
    new_base_dir / 'validation', # 指定驗證資料的目錄。
    image_size=(180,180), # 將圖片調整為 180x180 像素。
    batch_size=32 # 設定每個批次的圖片數量為 32。
)

test_dataset = image_dataset_from_directory( # 載入測試資料集。
    new_base_dir / 'test', # 指定測試資料的目錄。
    image_size=(180,180), # 將圖片調整為 180x180 像素。
    batch_size=32 # 設定每個批次的圖片數量為 32。
)

... Found 2000 files belonging to 2 classes.
Found 1000 files belonging to 2 classes.
Found 2000 files belonging to 2 classes.
```


以少量資料集從頭訓練 一個卷積神經網路

初始訓練結果與問題診斷

訓練方式：

- 使用 validation data 監控模型表現
- 搭配 ModelCheckpoint：只保留驗證表現最好的模型

```
[12] 2分 鐘
from tensorflow import keras # 從 TensorFlow 導入 Keras 模組。

callback = [ # 定義回呼函數列表。
    keras.callbacks.ModelCheckpoint( # ModelCheckpoint 回呼函數用於在訓練過程中儲存模型。
        filepath='convnet_from_scratch.keras', # 指定儲存模型的路徑和檔案名。
        save_best_only=True, # 只儲存驗證損失最小（最佳）的模型。
        monitor='val_loss')] # 監控驗證損失（validation loss）來判斷模型好壞。

history = model.fit( # 訓練模型。
    train_dataset, # 指定訓練資料集。
    epochs=30, # 設定訓練週期為 30。
    validation_data=validation_dataset, # 指定驗證資料集，用於在每個週期結束後評估模型性能。
    callbacks=callback # 傳入回呼函數列表。
)
```

... Epoch 1/30
63/63 ————— 15s 137ms/step - accuracy: 0.5161 - loss: 0.7072 - val_accuracy: 0.5000 -
Epoch 2/30

結果觀察

測試準確度是71%，由於訓練樣本較少，過度配適成為訓練時的首要顧慮因素。

```
[22] 4 秒
test_model = keras.models.load_model('convnet_from_scratch.keras') # 載入之前訓練的模型。

test_loss, test_acc = test_model.evaluate(test_dataset) # 在測試資料集上評估模型。
print(f'Test accuracy: {test_acc:.3f}') # 印出測試準確度。

63/63 ----- 4s 50ms/step - accuracy: 0.6942 - loss: 0.5930
Test accuracy: 0.710
```

還有一個方法可以避免過度配適：資料擴增法.....

以少量資料集從頭訓練一個卷積神經網路

解法一：資料擴增

在不增加新資料的前提下，透過隨機變換產生「不同樣貌的訓練樣本」，以增加資料多樣性，降低模型記憶特定樣本的風險。

使用的擴增方式

實作上，只要在模型的一開始加入資料擴增層就可以達到“資料擴增”這個目標。

定義擴增層：

```
from tensorflow import keras # 從 TensorFlow 導入 Keras 模組。
from tensorflow.keras import layers # 從 tensorflow.keras 導入 layers 模組。

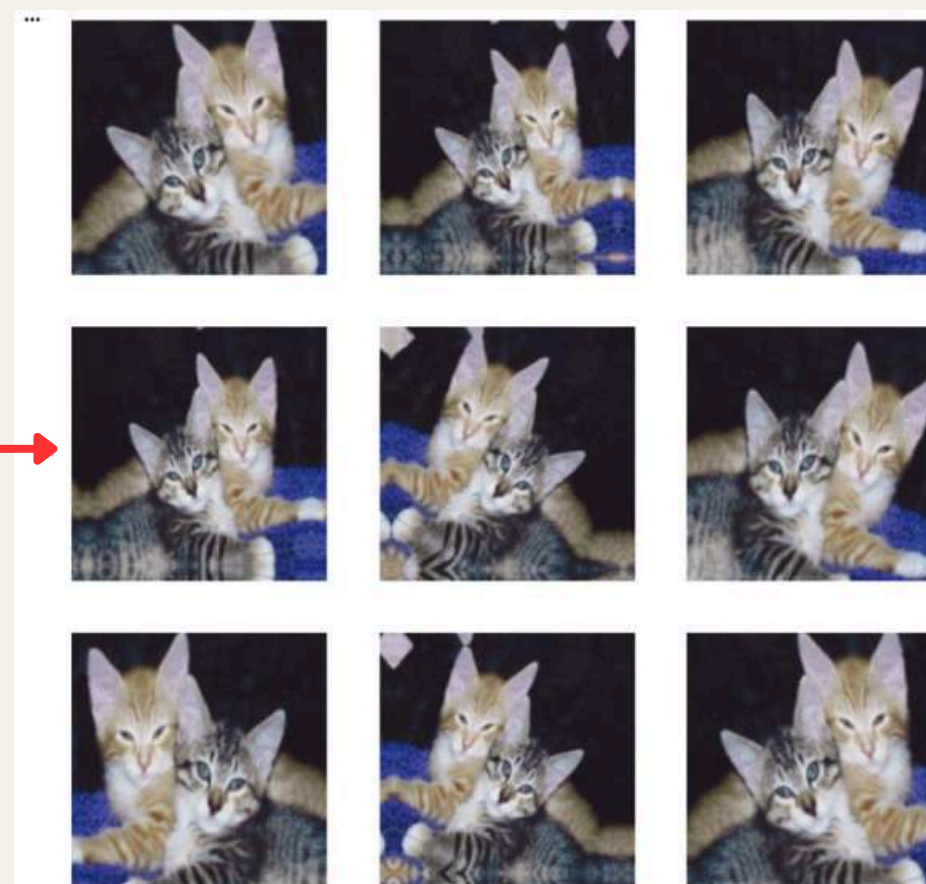
data_augmentation = keras.Sequential( # 定義資料擴增層的序列。
    [
        layers.RandomFlip('horizontal'), # 水平翻轉圖片。
        layers.RandomRotation(0.1), # 隨機旋轉圖片，角度範圍為 +/- 0.1 弧度。
        layers.RandomZoom(0.2), # 隨機縮放圖片，縮放因子範圍為 +/- 0.2。
    ]
)
```

- RandomFlip（水平翻轉）
- RandomRotation（小角度旋轉）
- RandomZoom（隨機縮放）

檢視經過“資料擴增”後的影像

```
[30]
import matplotlib.pyplot as plt # 導入 Matplotlib 庫的 pyplot 模組。

plt.figure(figsize=(10,10)) # 創建一個新的圖表，並設定圖表大小。
for images, _ in train_dataset.take(1): # 從訓練資料集中取出一個批次的圖片。
    for i in range(9): # 遍歷前 9 張圖片。
        augmented_images = data_augmentation(images) # 對圖片執行數據增強。
        ax = plt.subplot(3,3,i+1) # 在 3x3 的網格中創建子圖。
        plt.imshow(augmented_images[0].numpy().astype('uint8')) # 顯示增強後的圖片。
        plt.axis('off') # 關閉座標軸。
plt.show() # 顯示圖片。
```



以少量資料集從頭訓練 一個卷積神經網路

進階防過擬合：Dropout

因擴增影像仍高度相關，模型仍可能記住模式。因此在分類器前加入 Dropout，隨機關閉部分神經元，降低神經元間的共適應

```
[20] 0 秒
from tensorflow import keras # 從 TensorFlow 導入 Keras 模組。
from tensorflow.keras import layers # 從 tensorflow.keras 導入 layers 模組。

input_layer = keras.Input(shape=(180,180,3)) # 定義模型的輸入層，指定輸入圖片的形狀。
x = data_augmentation(input_layer) # 將輸入圖片傳遞給數據增強層。
x = layers.Rescaling(1./255)(x) # 將像素值縮放到 0-1 範圍。
x = layers.Conv2D(filters=32, kernel_size=3, activation='relu')(x) # 第一個卷積層，使用 ReLU 啟動函數。
x = layers.MaxPooling2D(pool_size=2)(x) # 第一個最大池化層。
x = layers.Conv2D(filters=64, kernel_size=3, activation='relu')(x) # 第二個卷積層，使用 ReLU 啟動函數。
x = layers.MaxPooling2D(pool_size=2)(x) # 第二個最大池化層。
x = layers.Conv2D(filters=128, kernel_size=3, activation='relu')(x) # 第三個卷積層，使用 ReLU 啟動函數。
x = layers.MaxPooling2D(pool_size=2)(x) # 第三個最大池化層。
x = layers.Conv2D(filters=256, kernel_size=3, activation='relu')(x) # 第四個卷積層，使用 ReLU 啟動函數。
x = layers.Flatten()(x) # 展平操作。
x = layers.Dropout(0.5)(x) # Dropout 層，以 0.5 的機率隨機關閉神經元，防止過擬合。
outputs = layers.Dense(1, activation='sigmoid')(x) # 輸出層，一個神經元，使用 Sigmoid 啟動函數。
model = keras.Model(inputs=input_layer, outputs=outputs) # 創建 Keras 模型。

model.compile(loss='binary_crossentropy', # 配置模型，損失函數為二元交叉熵。
              optimizer='rmsprop', # 優化器為 RMSprop。
              metrics=['accuracy']) # 評估指標為準確度。
```

改良後結果與比較

測試準確率提升至 83.3%

```
[36] 2 秒
from tensorflow import keras # 從 TensorFlow 導入 Keras 模組。
import os # 導入 os 模組。

model_filepath = 'convnet_from_scratch_with_augmentation.keras' # 定義模型檔案路徑。

if os.path.exists(model_filepath): # 檢查模型檔案是否存在。
    test_model = keras.models.load_model(model_filepath) # 載入最佳模型。
    test_loss, test_acc = test_model.evaluate(test_dataset) # 在測試資料集上評估模型。
    print(f'Test accuracy: {test_acc:.3f}') # 印出測試準確度。
else: # 如果檔案不存在。
    print(f'Error: Model file {model_filepath} not found.') # 印出錯誤訊息。
    print("Please ensure the training cell (cell 5k7lmQoYfZ7e) was executed successfully to save the model.") # 提醒用戶。

... 63/63 ----- 2s 30ms/step - accuracy: 0.8351 - loss: 0.4590
Test accuracy: 0.833
```

過度配適情況明顯改善，但從零訓練 CNN 在小資料集上仍有限制，很難再
向上突破..... → 需要引入 預訓練模型

使用預先訓練好的模型

使用預訓練模型的兩種策略

- 定義：事先以大量資料集訓練過的優秀模型
- 核心優勢：站在巨人的肩膀上，利用已學習的特徵知識

ImageNet資料集介紹

- 規模：140萬個標註影像
- 多樣性：1000個不同類別
- 內容：包含多種動物類別，如不同品種的貓狗等
- 地位：深度學習領域的重要基準（benchmark）資料集

VGG16神經網路

- 來源：已在Keras中預先安裝
- 訓練基礎：使用ImageNet訓練完成
- 架構特色：深層卷積神經網路，具有強大的特徵提取能力

```
[21]  
✓ 0 秒  
  
import keras  
# 這一行導入了 Keras 深度學習函式庫，讓我們可以使用其中的功能，例如預訓練模型。  
  
conv_base = keras.applications.vgg16.VGG16(  
    # 這一行初始化了一個 VGG16 卷積神經網路模型。VGG16 是一個在 ImageNet 資料集上預訓練過的經典模型。  
    weights='imagenet',  
    # 這個參數指定了模型要加載的權重。'imagenet' 表示使用在 ImageNet 資料集上訓練好的權重，這是一個大型的圖像分類資料集。  
    include_top=False,  
    # 這個參數設定為 False 意味著我們不包括模型的頂部（即分類層）。當我們將預訓練模型作為特徵提取器用於新的分類任務時，通常會這麼做，因為我們需要添加自定義的分類器。  
    input_shape=(180, 180, 3))  
# 這個參數定義了模型輸入圖像的形狀。這裡設定為 (180, 180, 3) 表示輸入圖像的高度為 180 像素，寬度為 180 像素，3 表示圖像有三個顏色通道（紅、綠、藍）。  
  
Downloading data from https://storage.googleapis.com/tensorflow/keras-applications/vgg16/vgg16\_weights\_tf\_dim\_ordering\_tf\_kernels\_notop.h5  
58889256/58889256 ————— 0s 0us/step
```


使用預先訓練好的模型

策略一：特徵萃取 (Feature Extraction)

- 做法：將預訓練模型當作「特徵提取器」
- 原理：凍結整個卷積基底，只訓練自定義的分類層

沒有資料擴增的快速特徵萃取

1. 使用 `preprocess_input()` 預處理圖像
2. 呼叫 `conv_base.predict()` 萃取特徵（輸出為 NumPy 陣列）
3. 在萃取的特徵上訓練密集分類層

搭配資料擴增的特徵萃取

- 關鍵技術：凍結 (Freezing)
- 凍結定義：在訓練期間禁止更新權重
- 實作方法：設定 `trainable=False`

```
[29] ✓ con_base = keras.applications.vgg16.VGG16(  
      weights='imagenet',  
      include_top=False)  
      con_base.trainable = False
```

最終成果

- 測試準確度：97.9%
- 效果提升：過度配適情況明顯改善

```
[59] ✓ 6 秒 test_model = keras.models.load_model(  
      "feature_extraction_with_data_augmentation.keras")  
      test_loss, test_acc = test_model.evaluate(test_dataset)  
      print(f"Test accuracy: {test_acc:.3f}")  
      ... 63/63 ----- 6s 91ms/step - accuracy: 0.9740 - loss: 2.5900  
      Test accuracy: 0.979
```


使用預先訓練好的模型

策略二：微調（Fine-tuning）

- 與特徵萃取的差異：不是凍結整個卷積基底，而是解凍「頂部」某些層
- 目的：讓預訓練模型適應新的特定任務

實作細節

1. 凍結：block4_pool之前的所有層
2. 解凍：block5_conv1、block5_conv2、block5_conv3

訓練技巧

- 優化器：RMSProp
- 學習率：非常低
- 原因：避免損害預訓練權重的表示法

最終成果

- 測試準確度：98%
- 顯著提升：相較於從零訓練或僅特徵萃取

```
[38] 6 秒
✓
model = keras.models.load_model("fine_tuning.keras")
test_loss, test_acc = model.evaluate(test_dataset)
print(f"Test accuracy: {test_acc:.3f}")

63/63 ----- 6s 92ms/step - accuracy: 0.9765 - loss: 1.9686
Test accuracy: 0.980
```


11 文字資料的深度學習

主講：張庭語

1 準備文字資料

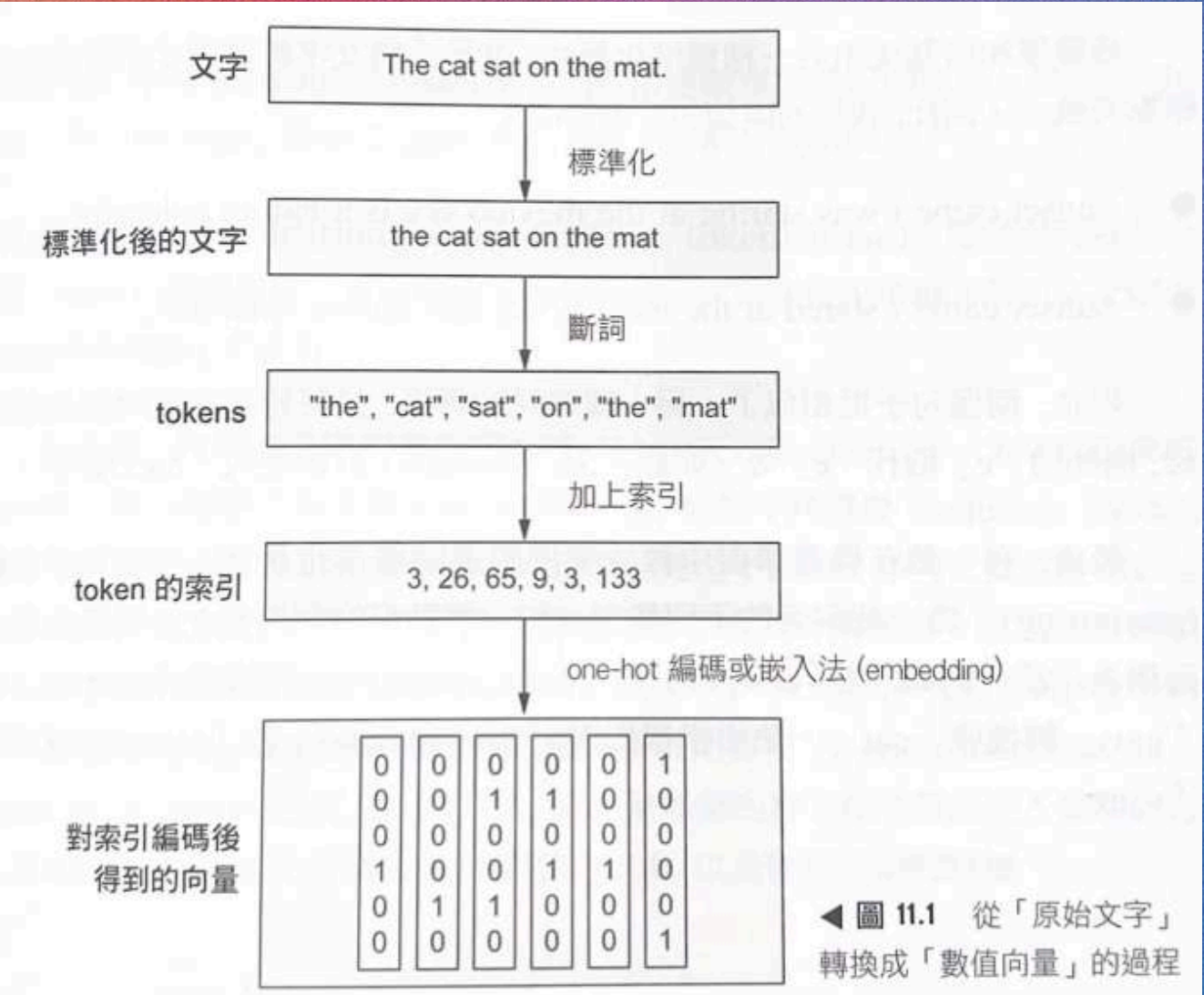
2 表示單字組的兩種方法

3 Transformer 架構

相關檔案

- [Google Colab](#)
- [Google Docs](#)

準備文字資料



文字向量化

Step 1: 文字標準化 (Text Normalization)

為了提升模型泛化能力，因此要：全部轉小寫、移除標點符號、統一格式

Step 2: 斷詞 (Tokenization)

三種方式：

- **單字層級**：以空格為單位，可搭配子詞拆解 (star + ing)，常用於序列模型 (RNN、Transformer)
 - **N-gram**：連續 N 個單字為一組，常用於詞袋模型 (不考慮順序)
 - **字元層級**：每個字母都是一個 token，少見，只在特殊任務出現
- 是否保留單字順序，會決定模型類型。

Step 3: 建立索引 (Indexing)

斷詞處理後，就要將每個Token編碼成數值表示法，經常使用的Token有兩種，分別是：OOV token 以及 遮罩token

OOV token	代表「這裡有一個無法辨識的單字」
遮罩token	代表「請忽略我，我不是一個單字」，遮罩token是用來填補序列資料的空缺，使序列資料的長度相等。 Ex：將序列 [5, 7, 124 ,4, 89] 與 [8, 34, 21] 變成一批次的資料： [[5, 7, 124 ,4, 89] [8, 34, 21 , 0, 0]] ps：較短的序列後方要補0，使其長度與最長的序列長度一樣長。

表示單字組的兩種方法

將單字視為一組集合：詞袋法

核心概念：只在乎「出現過什麼」，不在乎「怎麼排」 → 完全不管順序

實作內容：使用 TextVectorization，將文字轉成multi-hot（二元）向量

結果與觀察：效果不差（≈88%）

```
480/480 ————— 6s 15ms/step - accuracy: 0.9519 - loss: 0.1533 -  
Epoch 10/10  
400/400 ————— 5s 13ms/step - accuracy: 0.9544 - loss: 0.1534 -  
782/782 ————— 16s 20ms/step - accuracy: 0.8823 - loss: 0.2983  
Test acc: 0.883
```

局部順序補強：N-gram（bigram）

核心概念：不看整句順序，但保留「局部搭配」

方法演進

1. Bigram + 二元編碼

- 準確率提升（≈89.5%）
- 證明「局部順序很重要」

「the cat sat on the mat」就會變成：
{ the, the cat, cat, cat sat, sat, sat on, on, on the, the mat, mat }

```
400/400 ————— 6s 14ms/step - accuracy: 0.9716 - loss:  
Epoch 10/10  
400/400 ————— 5s 12ms/step - accuracy: 0.9717 - loss:  
782/782 ————— 17s 21ms/step - accuracy: 0.8924 - loss:  
Test acc: 0.895
```

2. TF-IDF bigram

- 解決常見字（the, is, are）過度影響
- 透過權重降低過度配適風險

藉由N-gram或是每個單字出現的次數，在表示法上再新增一些資訊：
{ the:2 , the cat:1 , cat: 1, cat sat: 1, sat: 1, sat on: 1, on: 1, on the: 1, the mat: 1, mat: 1 }

人工設計特徵，效果不錯。但現在要讓模型自己學順序！

表示單字組的兩種方法

將單字作為序列處理：序列模型

核心概念：不幫模型做特徵工程，直接讓模型「看單字序列」

三個關鍵步驟：用整數序列表示句子 → 把整數轉成向量 → 用能建模前後關係的模型（RNN / LSTM）

one-hot + LSTM

- 兩個問題：訓練非常慢、向量維度極高
- 準確率達84.4%

```
Epoch 8/10  
625/625 ————— 27s 43ms/step - accuracy: 0.9525 - loss: 0.1475 - val_accuracy: 0.8798  
Epoch 9/10  
625/625 ————— 44s 47ms/step - accuracy: 0.9666 - loss: 0.1123 - val_accuracy: 0.8754  
Epoch 10/10  
625/625 ————— 27s 43ms/step - accuracy: 0.9725 - loss: 0.0954 - val_accuracy: 0.8754  
782/782 ————— 15s 18ms/step - accuracy: 0.8450 - loss: 0.3631  
Test acc: 0.844
```

結果較使用unigram差，更好得方法是「使用詞嵌入」

詞嵌入（Word Embedding）

- one-hot：稀疏、高維、沒語意距離
- embedding：低維、連續、語意結構化

Embedding的本質

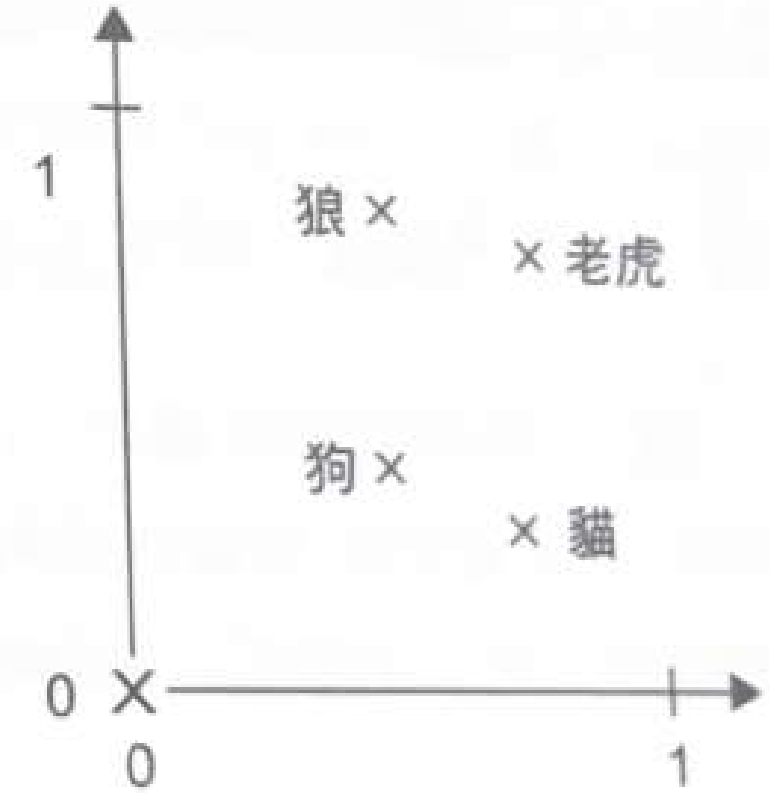
- 把單字映射到幾何空間
- 相似詞距離近、方向有意義

成效

- 訓練速度大幅提升
- 準確率與 one-hot LSTM 相近

```
625/625 ————— 41s 45ms/step - accuracy: 0.9704 - loss: 0.0690 -  
Epoch 9/10  
625/625 ————— 29s 46ms/step - accuracy: 0.9831 - loss: 0.0501 -  
Epoch 10/10  
625/625 ————— 40s 45ms/step - accuracy: 0.9869 - loss: 0.0378 -  
782/782 ————— 16s 20ms/step - accuracy: 0.8737 - loss: 0.3038  
Test acc: 0.875
```

→ 實務上最常用的做法



▲ 圖 11.3 詞嵌入空間的小案例

表示單字組的兩種方法

實務上高機率遇到的問題：Padding 與 Masking

為了 batch 訓練，所有句子要同長

```
[5]: 4 秒
✓ 秒
from tensorflow.keras import layers

max_length = 600
max_tokens = 20000
text_vectorization = layers.TextVectorization(
    max_tokens=max_tokens,
    output_mode="int",
    output_sequence_length=max_length,
)
text_vectorization.adapt(text_only_train_ds)

int_train_ds = train_ds.map(
    lambda x, y: (text_vectorization(x), y),
    num_parallel_calls=4
)
int_val_ds = val_ds.map(
    lambda x, y: (text_vectorization(x), y),
    num_parallel_calls=4
)
int_test_ds = test_ds.map(
    lambda x, y: (text_vectorization(x), y),
    num_parallel_calls=4
)
```

設定 max_length = 600

太長 → 截斷

太短 → 補 0 (padding)

問題

padding 補的 0 沒有語意，卻會被 RNN 當成有效輸入

解決方案：Masking

在 Embedding 層設定 mask_zero=True。系統會：標記 padding 位置、RNN 跳過這些時間步、loss 計算時忽略 padding

```
[5]: 5 秒
✓ 秒
inputs = keras.Input(shape=(None,), dtype="int64")
embedded = layers.Embedding(input_dim=max_tokens, output_dim=256, mask_zero=True)(inputs)

x = layers.Bidirectional(layers.LSTM(32))(embedded)
x = layers.Dropout(0.5)(x)
outputs = layers.Dense(1, activation="sigmoid")(x)
model = keras.Model(inputs, outputs)
model.compile(optimizer="rmsprop",
              loss="binary_crossentropy",
              metrics=["accuracy"])

model.summary()

callbacks = [
    keras.callbacks.ModelCheckpoint("embeddings_bidir_gru_with_masking.keras",
                                    save_best_only=True)
]

model.fit(int_train_ds, validation_data=int_val_ds, epochs=10, callbacks=callbacks)
model = keras.models.load_model("embeddings_bidir_gru_with_masking.keras")
print(f"Test acc: {model.evaluate(int_test_ds)[1]:.3f}")
```


表示單字組的兩種方法

預訓練詞嵌入（GloVe）

使用時機：資料量小、無法自己學好 embedding

實驗結果：在 IMDB 任務中效果不佳（≈87%）

```
[45] ✓ 分鐘
inputs = keras.Input(shape=(None,), dtype="int64")
embedded = embedding_layer(inputs)
x = layers.Bidirectional(layers.LSTM(32))(embedded)
x = layers.Dropout(0.5)(x)
outputs = layers.Dense(1, activation="sigmoid")(x)
model = keras.Model(inputs, outputs)

model.compile(optimizer="rmsprop",
              loss="binary_crossentropy",
              metrics=["accuracy"])

model.summary()

callbacks = [
    keras.callbacks.ModelCheckpoint("glove_embeddings_sequence_model.keras",
                                    save_best_only=True)
]

model.fit(int_train_ds, validation_data = int_val_ds, epochs=10, callbacks=callbacks)
model = keras.models.load_model("glove_embeddings_sequence_model.keras")
print(f"Test acc: {model.evaluate(int_test_ds)[1]:.3f}")
```

Layer (type)	Output Shape	Params #	Connected to
input_layer_11 (InputLayer)	(None, None)	0	-
embedding_8 (Embedding)	(None, None, 100)	2,000,000	input_layer_11[0]
not_equal_3 (NotEqual)	(None, None)	0	input_layer_11[0]
bidirectional_8 (Bidirectional)	(None, 64)	14,048	embedding_8[0][0] not_equal_3[0][0]
dropout_11 (Dropout)	(None, 64)	0	bidirectional_8[0]
dense_14 (Dense)	(None, 1)	65	dropout_11[0][0]

Total params: 2,034,113 (7.76 MB)
Trainable params: 34,113 (133.25 KB)
Non-trainable params: 2,000,000 (7.63 MB)

Epoch 1/10
625/625 — 83s 120ms/step - accuracy: 0.6148 - loss: 0.6482 - val_accuracy: 0.7908 - val_loss: 0.4635

Epoch 2/10
625/625 — 29s 47ms/step - accuracy: 0.7882 - loss: 0.4767 - val_accuracy: 0.8038 - val_loss: 0.4294

Epoch 3/10
625/625 — 30s 48ms/step - accuracy: 0.8138 - loss: 0.4125 - val_accuracy: 0.8230 - val_loss: 0.3952

Epoch 4/10
625/625 — 26s 42ms/step - accuracy: 0.8365 - loss: 0.3725 - val_accuracy: 0.7952 - val_loss: 0.4487

Epoch 5/10
625/625 — 30s 48ms/step - accuracy: 0.8484 - loss: 0.3535 - val_accuracy: 0.8536 - val_loss: 0.3438

Epoch 6/10
625/625 — 38s 42ms/step - accuracy: 0.8611 - loss: 0.3301 - val_accuracy: 0.8434 - val_loss: 0.3534

Epoch 7/10
625/625 — 29s 47ms/step - accuracy: 0.8707 - loss: 0.3109 - val_accuracy: 0.8550 - val_loss: 0.3424

Epoch 8/10
625/625 — 30s 48ms/step - accuracy: 0.8810 - loss: 0.2905 - val_accuracy: 0.8650 - val_loss: 0.3178

Epoch 9/10
625/625 — 26s 42ms/step - accuracy: 0.8876 - loss: 0.2761 - val_accuracy: 0.8532 - val_loss: 0.3022

Epoch 10/10
625/625 — 26s 42ms/step - accuracy: 0.8943 - loss: 0.2609 - val_accuracy: 0.8674 - val_loss: 0.3283

782/782 — 15s 19ms/step - accuracy: 0.8729 - loss: 0.3028

Test acc: 0.875

因為現有資料集中已經包含足夠的樣本，可以直接從頭學習適用於該任務的嵌入空間。

→ 資料集很小時，使用預訓練嵌入向量會很有幫助。

Transformer架構

Attention 是什麼？為什麼會革命性？

Attention 的本質：對一組特徵分配「重要性權重」

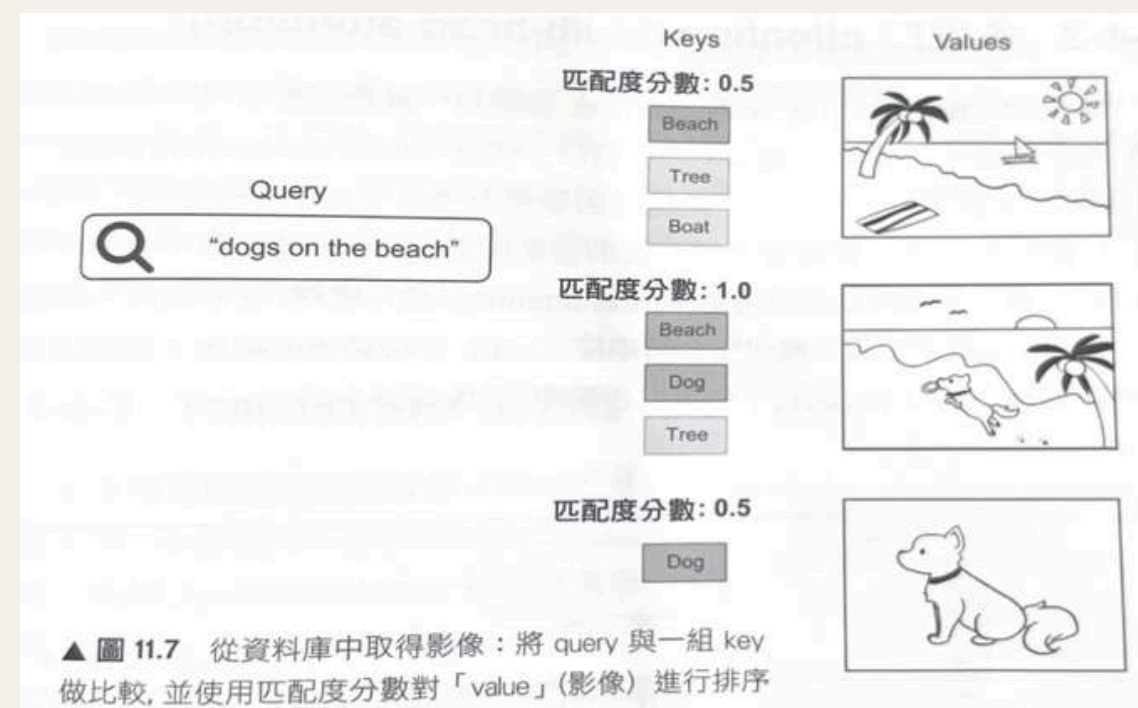
- 關聯高的 → 權重高
- 關聯低的 → 權重低

關鍵突破

不需要卷積、不需要循環，就能建立很強的序列模型，這就是論文
《Attention is All You Need》 的革命性之處

Self-Attention：模型怎麼「看懂上下文」？

- Self-Attention 讓模型在處理每一個 token 時，可以同時參考整個序列中其他 token 的資訊，Self-attention 可以想成搜尋引擎：每個 token 都會產生：Query、Key、Value，它會計算 token 與 token 之間的關聯程度，自動決定「現在應該關注誰」



Transformer架構

問題意識

如果只用一組 attention，模型一次只能關注一種關係...

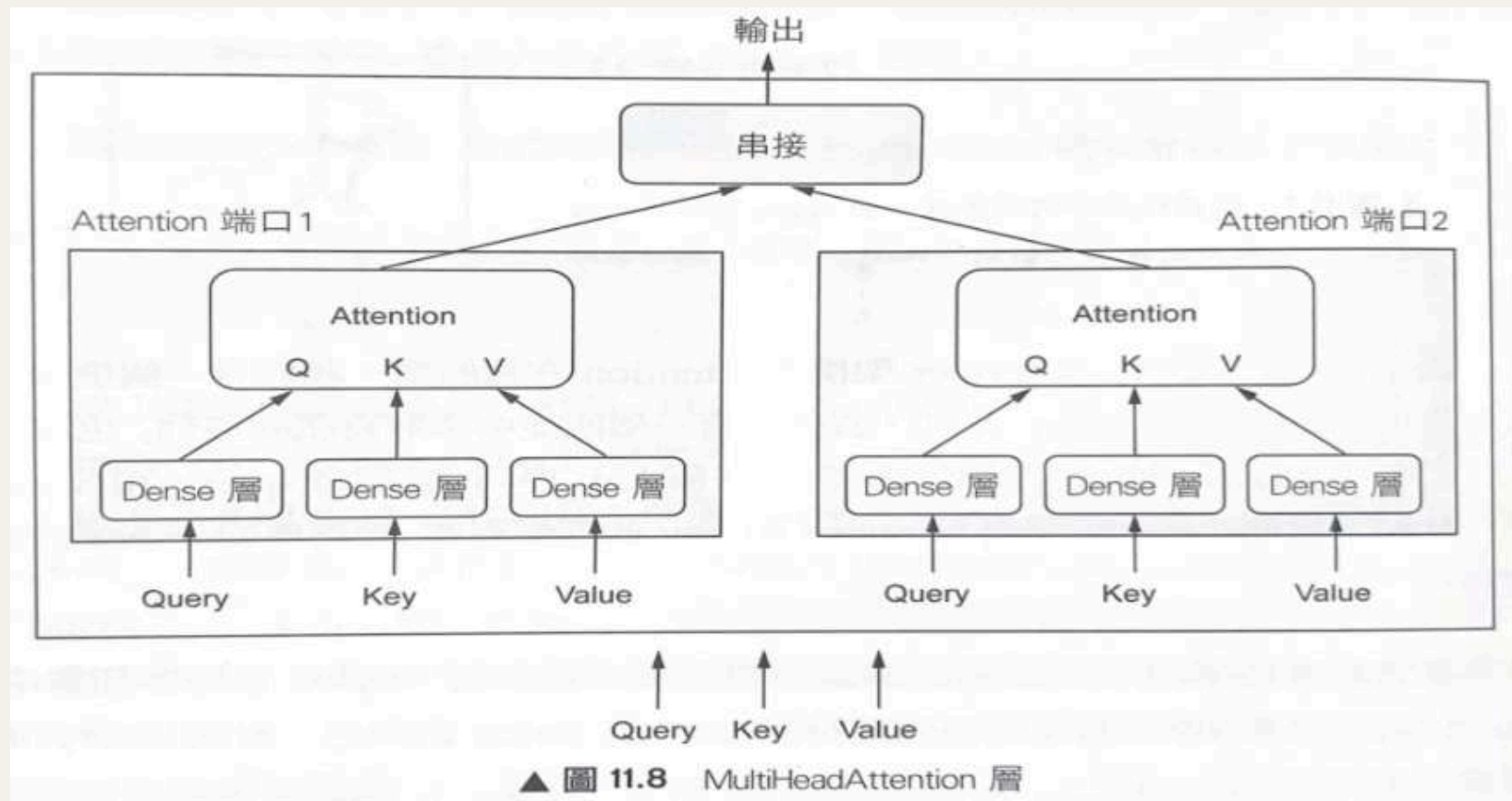
Multi-head 在做什麼？

把原始向量投影成多組 Q、K、V，每一組是一個 attention head。不同 head 專注於：

- 語意關係
- 句法結構
- 長距離依賴

最後：把所有 head 的輸出串接、整合，得到更全面的特徵表示

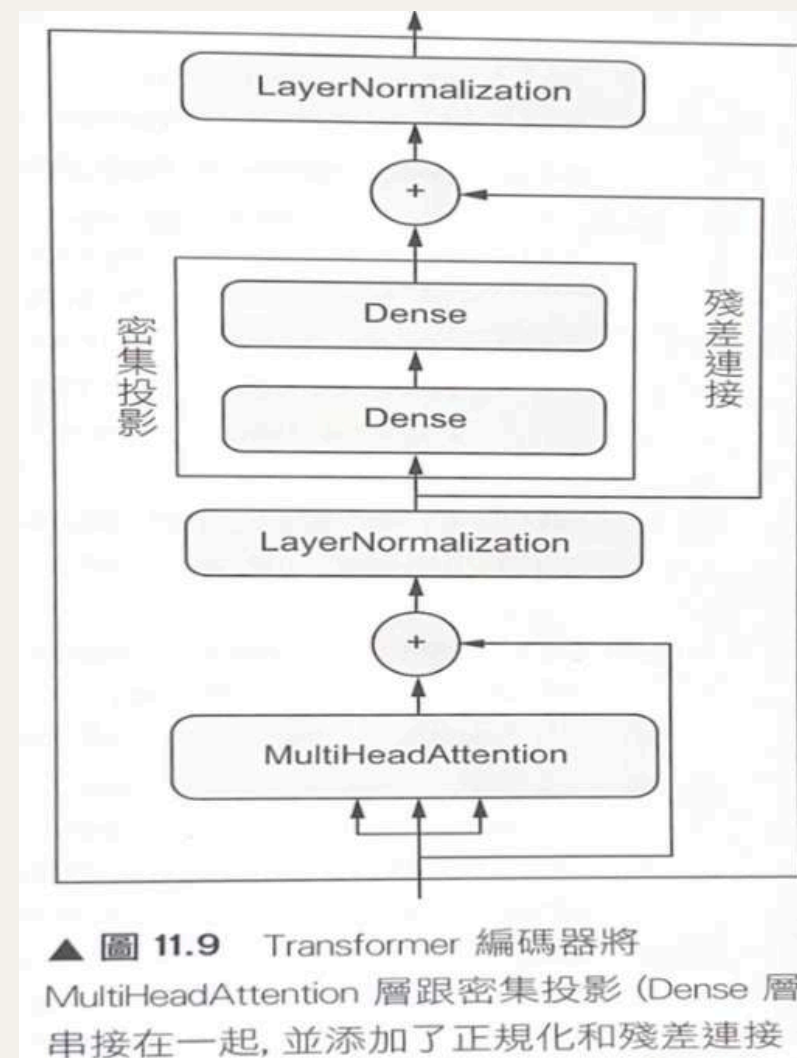
Multi-head attention 讓模型可以「同時從多個角度理解一句話」。



多個 attention head，各自關注不同語意關係，最後再整合

Transformer架構

Transformer 編碼器在做什麼？



一個 Transformer 編碼器包含：

1. 多頭 self-attention
2. 前饋神經網路 (Feed-forward)
3. 殘差連接 (Residual connection)
4. Layer Normalization

工作流程

- Attention：建立全句關聯
- Feed-forward：強化每個 token 的特徵
- Residual + LayerNorm：
 - 保留原始資訊
 - 穩定訓練流程

重複堆疊這個模組，就能建立深層 Transformer

實作 Transformer 做文字分類 (IMDB)

Transformer (無位置資訊) 約 86.5%，表現略低於 GRU。

```
625/625 ————— 57s 90ms/step - accuracy: 0.9739 - loss: 0.0778
Epoch 18/20
625/625 ————— 56s 89ms/step - accuracy: 0.9765 - loss: 0.0689
Epoch 19/20
625/625 ————— 54s 86ms/step - accuracy: 0.9779 - loss: 0.0588
Epoch 20/20
625/625 ————— 52s 84ms/step - accuracy: 0.9812 - loss: 0.0535
/usr/local/lib/python3.12/dist-packages/keras/src/layers/layer.py:421: UserWarning: 'build0' wa
warnings.warn(
782/782 ————— 15s 17ms/step - accuracy: 0.8602 - loss: 0.3123
Test acc: 0.865
```

Transformer 為什麼效果沒有想像中好？

Transformer架構

關鍵問題：Attention 本身「不知道順序」

核心限制：Attention 天生對順序不敏感，只看關聯，不知道「誰在前誰在後」。

解決方案：位置嵌入（Positional Embedding）

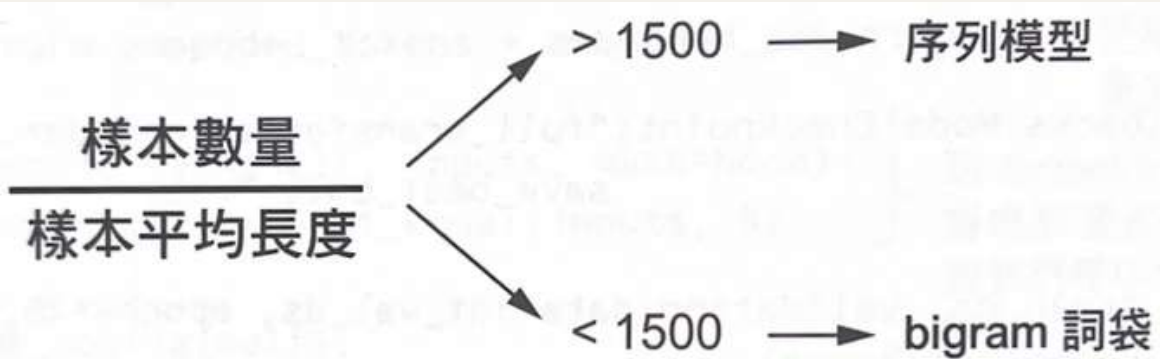
目的：把「單字在句子中的位置」告訴模型

- 兩種方法：使用週期函數（sin / cos）、學習式位置嵌入
- 作法：把舊的Embedding層換成「可感知位置」版本的Embedding層，也就是：PositionalEmbedding層

```
625/625 ————— 36s 57ms/step - accuracy: 0.9951 - loss: 0.0176 -  
Epoch 20/20  
625/625 ————— 35s 56ms/step - accuracy: 0.9967 - loss: 0.0120 -  
/usr/local/lib/python3.12/dist-packages/keras/src/layers/layer.py:421: UserWar  
nings.warn(  
/usr/local/lib/python3.12/dist-packages/keras/src/layers/layer.py:421: UserWar  
nings.warn(  
782/782 ————— 15s 11ms/step - accuracy: 0.8844 - loss: 0.2868  
Test acc:0.883
```

最後的測試準確度高達88.3%，證明了：單字順序的資訊在文字分類中的重要性。

何時該選擇序列模型，而非詞袋模型



▲ 圖 11.11 選擇文字分類模型的一種簡單方法：
計算「樣本數」與「樣本平均長度」之間的比率

Transformer架構

seq2seq的學習

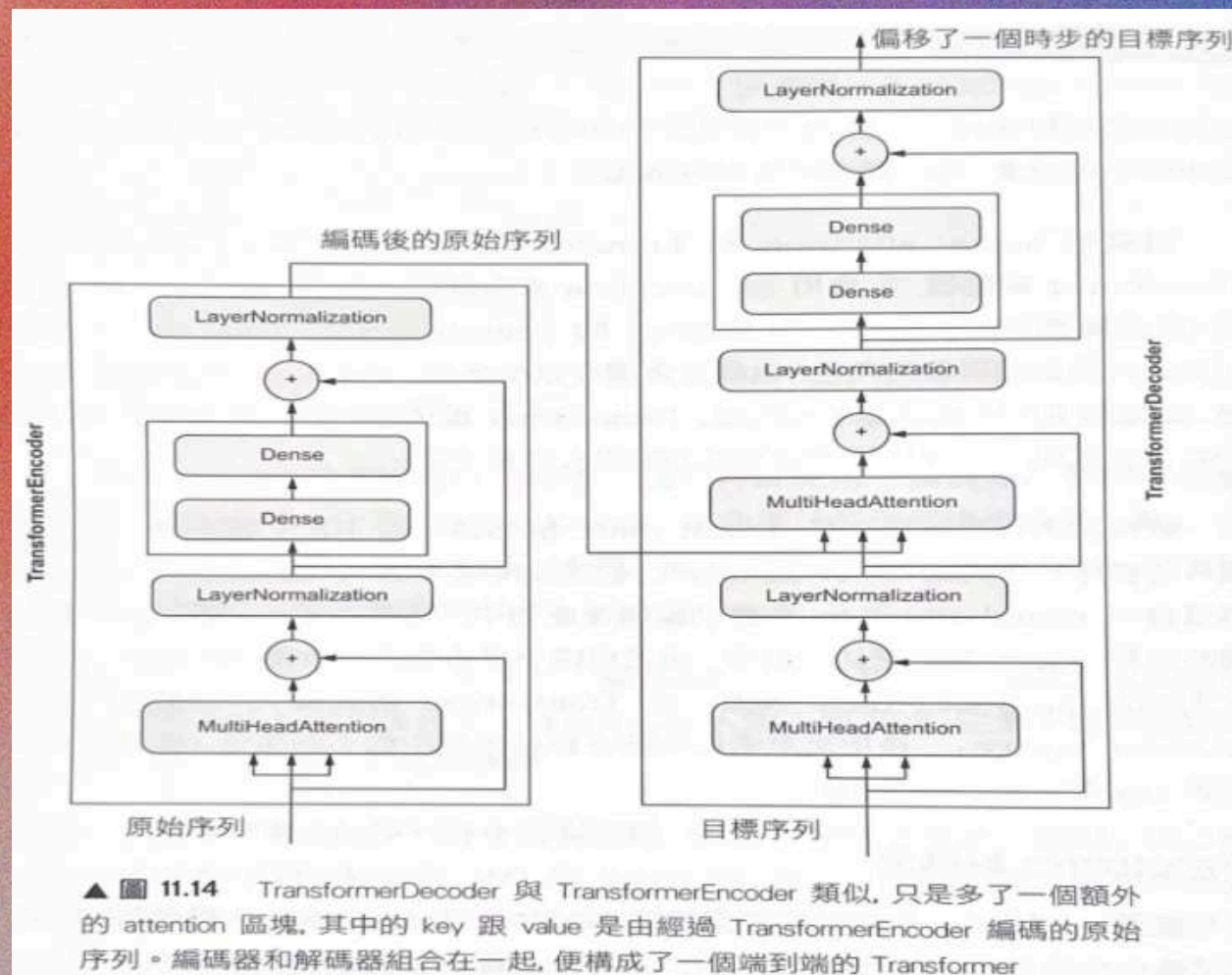
會把一個序列作為輸入(通常是一個句子或段落)，並將它轉換成不同的序列，EX：翻譯(原始語言轉目標語言)

用Transformer進行seq2seq的學習

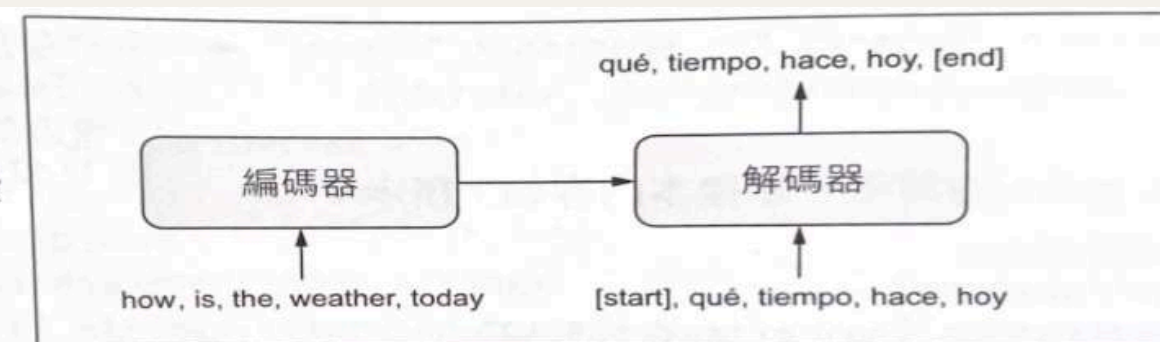
Transformer 解碼器與Transformer編碼器很像，解碼是在中間又多了MultiHeadAttation層跟LayerNormalization。

Transformer 解碼器

學習如何透過編碼向量，加上目標序列中的先前Token，來預測下一個位置的Token。



訓練階段



推論階段

